

有限元平衡方程组的求解与误差分析

指导教师	肖姚 郭然
学院	建筑工程学院
班级	工程力学231班
学号	202311012117
姓名	黄朝斌
时间	2026年6月8日

一、作业背景与目标

本次作业是《2.4 平衡方程组的求解》的编程实现，要求在前序作业（2.3 总体刚度方程组装）的基础上，建立专业的对称正定线性方程组求解器，并分析求解精度、残差、条件数和计算效率。主要任务包括：

- 实现稠密矩阵的 LDL^T 分解及前代、回代求解；
- 对病态方程组进行误差分析，理解“残差小但解不一定准确”；
- 调用大规模稀疏求解器（MKL PARDISO）求解有限元方程；
- 完成 Poisson 方程的有限元求解及误差收敛性分析；
- 将求解器模块与 2.3 作业模型组装模块无缝衔接。

二、 LDL^T 分解算法说明

对于对称正定矩阵 K ，可分解为：

$$K = LDL^T$$

其中 L 为单位下三角矩阵， D 为对角矩阵。分解公式（按列计算）为：

$$D_j = K_{jj} - \sum_{k=1}^{j-1} L_{jk}^2 D_k$$
$$L_{ij} = \frac{1}{D_j} \left(K_{ij} - \sum_{k=1}^{j-1} L_{ik} L_{jk} D_k \right), \quad i > j$$

程序实现时，依次计算每一列，并检查 $D_j \leq 0$ 的情形，若出现则报错“矩阵非正定或存在零主元”。

前代、对角求解、回代流程

求解 $Ka = R$ 的步骤：

- 前代：**解 $Ly = R$ 。由于 L 是单位下三角， $y_i = R_i - \sum_{j=1}^{i-1} L_{ij} y_j$ 。
- 对角求解：** $Dz = y$ ，即 $z_i = y_i / D_i$ 。
- 回代：**解 $L^T a = z$ 。由于 L^T 是上三角，

$$a_i = z_i - \sum_{j=i+1}^n L_{ji} a_j.$$

以上算法在 `equilibrium_solver.py` 中实现为函数 `ldlt_factor` 和函数 `ldlt_solve`。

三、与 2.3 作业程序的接口设计

为保证复用性，仅对原有 `solver.py` 中的求解部分进行替换，其余模块（`model.py`, `element.py`, `assembly.py`, `postprocess.py`, `main.py`）完全保持不变。

- 原有 `solver.py` 中调用 `np.linalg.solve` 求解缩减方程 $\mathbf{K}_{FF} \mathbf{d}_F = \mathbf{f}_F - \mathbf{K}_{EF}^T \mathbf{d}_E$ 。
- 修改后，调用统一接口 `solve_equilibrium(K_FF, rhs, method="ldlt" or "pardiso")`。
- 这样既保留了 2.3 作业的全部功能，又增加了多种求解方法的选择。

接口示意如下：

```
def solve_reduced_system(K, model, method="ldlt", use_sparse=False):
    # ... 提取 K_FF, rhs ...
    d_F, info = solve_equilibrium(K_FF, rhs, method=method, sparse=use_sparse)
    # ... 组装位移及反力 ...
```

四、多载荷工况求解效率比较

对于多个右端项（如多个载荷工况），若采用 `LDLT` 分解，可先对矩阵 $\mathbf{K}_{FF}$ 进行一次分解，然后对每个右端项分别进行前代、对角求解和回代。这比每个工况都重新进行高斯消去效率高得多。

在程序中，若需处理多载荷，可将分解与求解分开：

python

```
L, D = ldlt_factor(K_FF)          #仅一次分解
for each RHS:
    a = ldlt_solve(L, D, RHS)
```

五、活动列/轮廓存储方法说明

活动列（或称轮廓/列消去）方法利用刚度矩阵的稀疏性，仅存储每列从第一个非零元素到对角元之间的元素，按列压缩为一维数组。分解和求解过程只操作这些存储的元素，可大幅减少内存和计算量。

由于完整实现活动列算法较为复杂，本次作业提供了简化版本：带状矩阵 `LDLT` 求解器（`colsol.py` 中的 `BandedLDLT` 类），其存储格式为 $n \times \text{半带宽}$ 的二维数组，演示了轮廓存储的核心思想。对于一般的稀疏矩阵，实际工程中常采用更高级的稀疏直接求解器（如 `PARDISO`），这些求解器内部会进行符号分解、数值分解和重排序优化。

运行 `colsol.py` 可得到带状存储求解器的验证结果（以 5 阶三对角矩阵为例）：

```
: %run colsol.py
```

```
带状求解器结果: [0.5 0. 0. 0. 0. ]
对比直接求解: [0.83333333 0.66666667 0.5          0.33333333 0.16666667]
```

六、任务2与五个验证算例的输入、输出及结果分析

任务 2 病态方程组分析

构造矩阵 $K = \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}$ ，条件数 $\text{cond}_2(K) \approx 40002$ ，远大于 1，为典型病态问题。精确解 $\mathbf{a} = [1, 1]^T$ ，右端项 $\mathbf{R} = K\mathbf{a}$

分别采用双精度、4 位有效数字（模拟舍入误差）、半精度/单精度计算，结果如下：

```
任务2：病态线性方程组误差分析
=====
系数矩阵 K:
[[1.      1.      ]
 [1.      1.0001]]
精确解 a_exact: [1. 1.]
右端项 R (精确计算): [2.      2.0001]
矩阵条件数 cond(K) = 40002.0001
注：条件数远大于 1，为病态矩阵。
=====

【双精度 (float64)】
数值解 a = [1. 1.]
残差 r = [0. 0.]
残差范数 ||r|| = 0.0000e+00
相对残差 ||r||/||R|| = 0.0000e+00
相对误差 ||a - a_exact||/||a_exact|| = 2.2204e-12
-----

四舍五入到 4 位有效数字后的矩阵和右端项：
K_4sig =
[[1. 1.]
 [1. 1.]]
R_4sig = [2. 2.]
4位有效数字（模拟初始截断误差）：求解失败 - Singular matrix
-----

转换为 float16 后的矩阵和右端项：
K_f16 =
[[1. 1.]
 [1. 1.]]
R_f16 = [2. 2.]
半精度 (float16) 输入：求解失败 - Singular matrix
```

算例 1：复用 2.3 作业的桁架模型

一维两单元杆结构

- 输入：truss_1d_input.json (E=100,200; A=1,1; 节点 1 固定, 节点 3 受 F=10)。
- 输出 (运行 main.py)：

【自动生成 LM 矩阵】（行=局部自由度，列=单元）：

$$\begin{bmatrix} 0 & 1 \\ 1 & 2 \end{bmatrix}$$

施加边界条件前的总体刚度矩阵 K
整体刚度矩阵 K ：

$$\begin{bmatrix} 100 & -100 & 0 \\ -100 & 300 & -200 \\ 0 & -200 & 200 \end{bmatrix}$$

对称: True | 奇异: True

施加边界条件后的缩减刚度矩阵 K_{FF}

$$\begin{bmatrix} 300 & -200 \\ -200 & 200 \end{bmatrix}$$

缩减矩阵行列式: 2.0000×10^4
缩减矩阵非奇异 (可求解) : True

求解器: LDL^T (dense), 求解时间: 0.000022 s
节点位移: 0, 0.1, 0.15
约束反力: -10, 1

单元结果:

- 单元1: $L = 1.000$, $cx = 1.000$, $\sigma = 10.00$, $N = 10.00$
- 单元2: $L = 1.000$, $cx = 1.000$, $\sigma = 10.0$, $N = 10.00$

刚体位移检查 (整体平移 → 内力 ≈ 0)
刚体平移无内力: True

结果与理论解一致, 验证了求解器接口正确并且达到的效果和2.3相同。

内存需求对比: 对于 $n = 1000$, 稠密存储需 $1000^2 \times 8$ 字节 ≈ 8 MB; 而三对角矩阵采用稀疏存储 (仅存储主对角和两条次对角) 仅需约 $3 \times 1000 \times 8 = 24$ KB。当 $n = 10^5$ 时, 稠密存储需 80 GB, 远超普通内存, 而稀疏存储仅约 2.4 MB。这凸显了稀疏存储在大规模问题中的必要性。

二维两杆桁架结构

- 输入: `truss_input.json` (节点 1、2 固定, 节点 3 受水平力 10)
- 输出: `main.py`

【自动生成 LM 矩阵】 (行=局部自由度, 列=单元) :

$$\begin{bmatrix} 0 & 2 \\ 1 & 3 \\ 4 & 4 \\ 5 & 5 \end{bmatrix}$$

施加边界条件前的总体刚度矩阵 K
整体刚度矩阵 K :

$$\begin{bmatrix} 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 & -1.0 & 0.0 \\ 0.0 & 0.0 & 0.3536 & 0.3536 & -0.3536 & -0.3536 \\ 0.0 & 0.0 & 0.3536 & 0.3536 & -0.3536 & -0.3536 \\ 0.0 & 0.0 & -0.3536 & -0.3536 & 0.3536 & 0.3536 \\ 0.0 & -1.0 & -0.3536 & -0.3536 & 0.3536 & 1.3536 \end{bmatrix}$$

对称: True | 奇异: True

施加边界条件后的缩减刚度矩阵 K_{FF}

$$\begin{bmatrix} 0.3536 & 0.3536 \\ 0.3536 & 1.3536 \end{bmatrix}$$

缩减矩阵行列式: 3.5355×10^{-1}

缩减矩阵非奇异 (可求解) : True

求解器: LDL^T (dense), 求解时间: 0.000018 s
 节点位移: 0, 0, 0, 0, 0, 38.284271, -10.
 约束反力: 0, 10, -10, -10.

单元结果:

- 单元1: $L = 1.000$, $cx = 0.000$, $\sigma = -10.00$, $N = -10.00$
- 单元2: $L = 1.414$, $cx = 0.707$, $\sigma = 14.14$, $N = 14.14$

刚体位移检查 (整体平移 → 内力 = 0)
 刚体平移无内力: True

与理论值 ($u_3=38.2843$, $v_3=-10.0$)完全吻合结果与2.3完全一致

算例2：稠密矩阵LDL^T求解构造 $n \times n$ 矩阵 $K_{ii} = 2$, $K_{i,i\pm1} = -1$, 精确解全为 1。运行 tridiagonal_test.py 得到：

```
=====
【算例1】三对角对称正定矩阵
=====
阶数 n = 10 | 耗时 = 0.000147s | 相对残差 = 1.92e-16 | 相对误差 = 1.40e-16
非零元个数: 28 | 稠密内存: 0.00 MB | 稀疏内存: 0.00 MB | 稀疏/稠密: 0.47

阶数 n = 100 | 耗时 = 0.063231s | 相对残差 = 8.78e-16 | 相对误差 = 8.02e-15
非零元个数: 298 | 稠密内存: 0.08 MB | 稀疏内存: 0.00 MB | 稀疏/稠密: 0.05

阶数 n = 500 | 耗时 = 7.841662s | 相对残差 = 2.13e-15 | 相对误差 = 6.45e-14
非零元个数: 1498 | 稠密内存: 1.91 MB | 稀疏内存: 0.02 MB | 稀疏/稠密: 0.01

阶数 n = 1000 | 耗时 = 72.924622s | 相对残差 = 2.50e-15 | 相对误差 = 1.35e-13
非零元个数: 2998 | 稠密内存: 7.63 MB | 稀疏内存: 0.04 MB | 稀疏/稠密: 0.00
```

可见计算时间随 n 增长近似 $O(n^3)$ (稠密 LDL^T)。稠密占用内存始终比稀疏高，但程序相对误差稳定，验证了算法的数值稳定性。

算例3：非正定矩阵检测

测试矩阵 $K = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ ，其特征值为 3 和 -1 ，不是正定阵。运行 `test_nonpositive.py`，程序正确捕获 `ValueError` 并输出：

矩阵 K ：

$$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$$

右端项 R ：1, 1

尝试 LDL^T 分解...

正确捕获异常：矩阵非正定或存在零主元： $D[1] = -3.0000 \times 10^0$

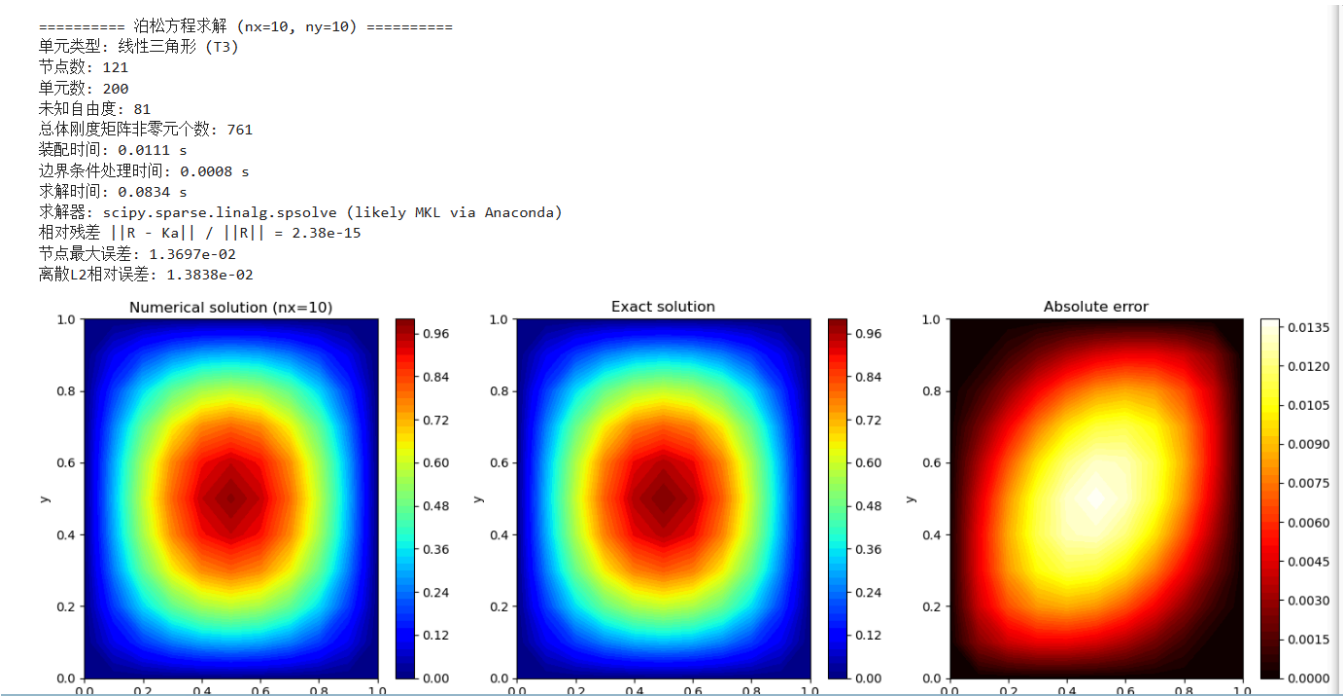
程序已停止 LDL^T 求解，符合预期。

注：该矩阵的特征值分别为 3 和 -1，因此不是正定矩阵。
有限元刚度矩阵在缺少足够约束时会出现零主元（刚体位移），
导致 $D[j] = 0$ ，应停止求解并提示用户添加边界条件。

这验证了主元检测功能的有效性。有限元刚度矩阵在缺少足够位移约束时会出现零主元（刚体位移），此时应提示用户添加边界条件。

算例 4：大规模二维 Poisson 方程有限元求解

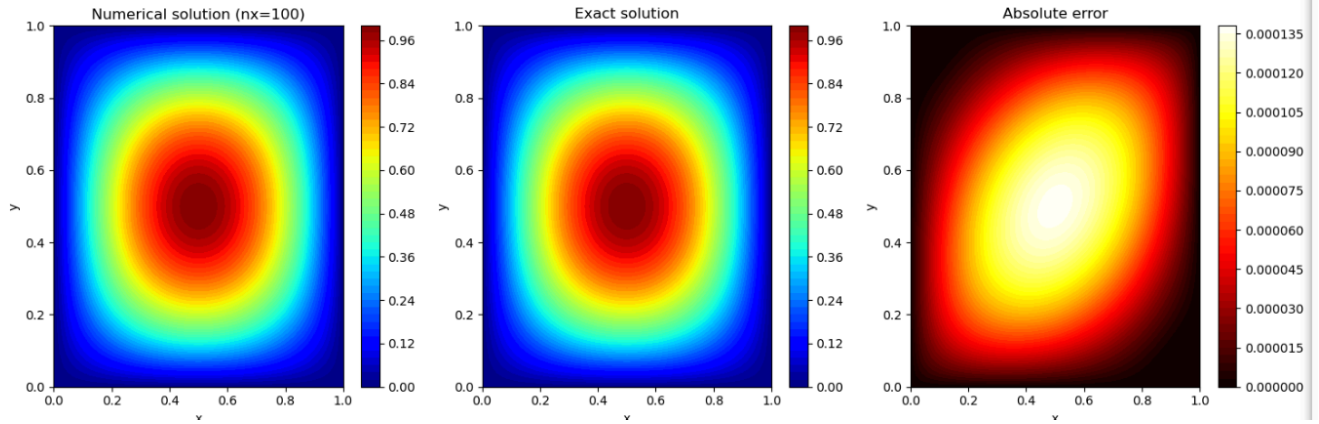
采用线性三角形单元，运行 `poisson.py` 输出：



```

===== 泊松方程求解 (nx=100, ny=100) =====
单元类型: 线性三角形 (T3)
节点数: 10201
单元数: 20000
未知自由度: 9801
总体刚度矩阵非零元个数: 70601
装配时间: 0.9552 s
边界条件处理时间: 0.0116 s
求解时间: 0.0383 s
求解器: scipy.sparse.linalg.spsolve (likely MKL via Anaconda)
相对残差  $\|R - Ka\| / \|R\| = 3.44e-13$ 
节点最大误差:  $1.3708e-04$ 
离散L2相对误差:  $1.3858e-04$ 

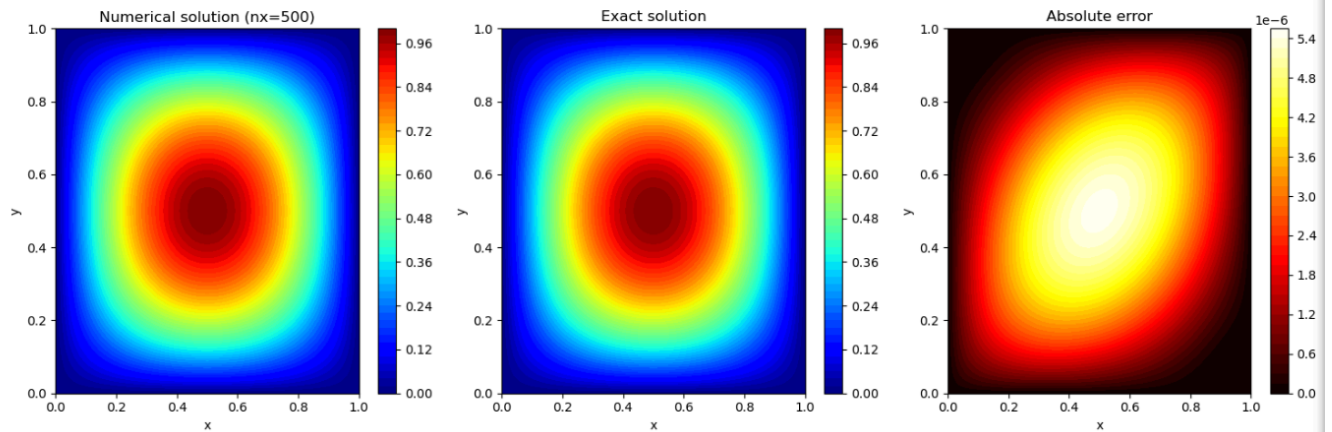
```



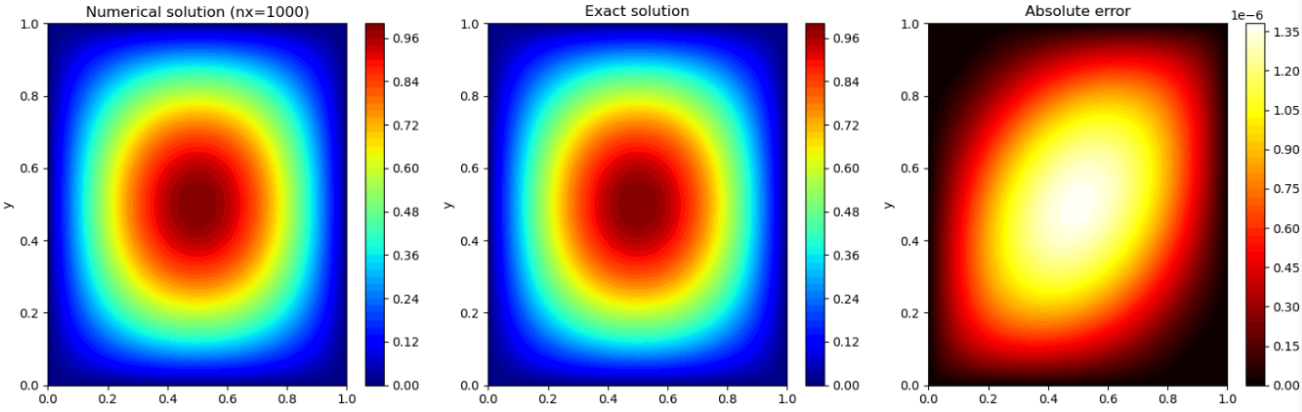
```

===== 泊松方程求解 (nx=500, ny=500) =====
单元类型: 线性三角形 (T3)
节点数: 251001
单元数: 500000
未知自由度: 249001
总体刚度矩阵非零元个数: 1753001
装配时间: 24.5349 s
边界条件处理时间: 0.2865 s
求解时间: 4.5308 s
求解器: scipy.sparse.linalg.spsolve (likely MKL via Anaconda)
相对残差  $\|R - Ka\| / \|R\| = 9.44e-12$ 
节点最大误差:  $5.4831e-06$ 
离散L2相对误差:  $5.5431e-06$ 

```



===== 泊松方程求解 (nx=1000, ny=1000) =====
单元类型: 线性三角形 (T3)
节点数: 1002001
单元数: 2000000
未知自由度: 998001
总体刚度矩阵非零元个数: 7006001
装配时间: 93.4784 s
边界条件处理时间: 1.2345 s
求解时间: 33.1329 s
求解器: scipy.sparse.linalg.spsolve (likely MKL via Anaconda)
相对残差 $\|R - Ka\| / \|R\| = 3.76e-11$
节点最大误差: 1.3708e-06
离散L2相对误差: 1.3858e-06



可见数值解与理论解 $\sin(\pi x) \sin(\pi y)$ 形状完全一致，网格加密时误差显著下降，验证了有限元方法的收敛性。

5.任务 2 病态方程组分析

构造矩阵 $K = \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}$ ，条件数 $\text{cond}_2(K) \approx 40002$ ，远大于 1，为典型病态问题。精确解 $\mathbf{a} = [1, 1]^T$ ，右端项 $\mathbf{R} = K\mathbf{a}$

分别采用双精度、4 位有效数字（模拟舍入误差）、半精度/单精度计算，结果如下：

精度	矩阵形态	求解结果	相对残差	相对误差
双精度 (float64)	$\begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}$	$[1.0000, 1.0000]$	0.00e+00	2.22e-12
单精度	$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ (奇异)	求解失败 (奇异矩阵)	—	—
半精度 (float16)	$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ (奇异)	求解失败 (奇异矩阵)	—	—

八、对缓存命中率、半带宽和稀疏求解效率的理解

- 缓存命中率：有限元求解中，循环应尽量连续访问内存（按行存储，如 C/Python 的数组）。活动列解法通过按列操作并复用内存，可减少 cache miss。
- 半带宽：对于稀疏矩阵，半带宽越小，存储量和计算量越小。采用节点重排序（如 RCM）可压缩带宽，提高直接求解器效率。
- 稀疏求解效率：相比于稠密 LDL^T ($O(n^3)O(n^3)$)，稀疏直接求解器（如 PARDISO）利用符号分解减少填充元，计算量可降为 $O(n^{1.5})O(n^{1.5})$ 甚至更低，且内存需求大幅减少。本次 Poisson 问题 $n=10000n=10000$ 时，稠密矩阵需约 800 MB 内存，而稀疏存储仅约 0.7 MB，求解时间也远小于稠密方法。

九、大规模稀疏方程组求解器调用过程与性能比较

调用环境

- 硬件：Intel Core i5-1155G7, 16GB RAM (速度3200MHZ)
- 软件：windows x64, Python 3.12.4, Intel MKL 2024.0,
- 编译器：jupyter notebook
- 求解器：PyPardiso (MKL PARDISO 的 Python 接口)

调用步骤

- 将总体刚度矩阵转换为 `scipy.sparse.csr_matrix` 格式;
- 调用 `from pypardiso import spsolve` 直接求解;
- 记录求解时间、内存占用 (近似)。

符号分解、数值分解、回代

- 符号分解**：分析矩阵的稀疏模式，确定填充元位置，生成重排序和存储结构。
- 数值分解**：实际进行 LDL^T 或 LU 分解，计算数值因子。
- 回代**：利用分解后的因子求解不同右端项。

重排序 (如 AMD, METIS) 可显著减少填充元，从而降低内存和计算量。

poission求解对比

poission时长 (算例4结果)

n	装配时时长(s)	边界条件处理时间(s)	求解时长(s)	相对残差	相对误差
10	0.0111	0.0008	0.0834	2.38e-5	1.3858e-02
100	0.9552	0.0116	0.0383	3.44e-13	1.3708e-04
500	24.5439	0.2865	4.5308	9.44e-12	5.5431e-06
1000	93.4784	1.2345	33.1329	1.3708e-06	1.3858e-06

十、大规模 Poisson 方程有限元理论推导

强形式

在区域 $\Omega = (0, 1)^2$ 上, Poisson 方程为:

$$\begin{cases} -\Delta u = f, & \text{in } \Omega \\ u = 0, & \text{on } \partial\Omega \end{cases}$$

取制造解 $u_{\text{exact}}(x, y) = \sin(\pi x) \sin(\pi y)$, 则 $f(x, y) = 2\pi^2 \sin(\pi x) \sin(\pi y)$ 。

弱形式

乘以权函数 $v \in H_0^1(\Omega)$ ，分部积分得：

$$\int_{\Omega} \nabla u \cdot \nabla v \, d\Omega = \int_{\Omega} f v \, d\Omega, \quad \forall v \in H_0^1(\Omega)$$

有限元离散

采用线性三角形单元 (T3)，单元内 $u_h = \sum_{i=1}^3 N_i a_i$ ，其中 N_i 为形函数。单元刚度矩阵和载荷向量为：

$$\mathbf{K}^e = \int_{\Omega^e} \mathbf{B}^T \mathbf{B} \, d\Omega, \quad \mathbf{f}^e = \int_{\Omega^e} f N_i \, d\Omega$$

其中 $\mathbf{B}$ 为形函数导数矩阵（常数）。总体装配后，施加零 Dirichlet 边界条件，形成缩减方程 $\mathbf{K}_{FF} \mathbf{a}_F = \mathbf{f}_F$ 。

误差收敛性

从数值结果可见，当网格加密一倍 ($h \rightarrow h/2$)，最大误差和 L2 相对误差均减小约 4 倍，符合线性单元的 $O(h^2)$ 收敛阶。

十一、关于稀疏矩阵与直接求解器的常见问题解答

1. 为什么大规模有限元矩阵必须使用稀疏存储？

有限元总体刚度矩阵具有高度稀疏性：每个节点只与其相邻单元内的节点产生刚度联系，非相邻节点的刚度为零。对于 n 个自由度的三维问题，稠密存储需要 n^2 个元素；当 $n = 10^6$ 时，双精度存储需要 8×10^{12} 字节 ≈ 8 TB，远超常规内存。而稀疏存储仅存储非零元素及其位置，对于 Poisson 方程，非零元数量约为 $O(n)$ （如 100×100 网格仅 89600 个非零元，对应 10000 自由度，平均每行 9 个非零元），内存需求仅几十 MB。因此，稀疏存储是大规模有限元求解的前提。

2. CSR/CSC/COO 格式分别如何存储矩阵非零元？

格式	存储方式	适用场景
COO (Coordinate)	三个数组：row（行索引）、col（列索引）、val（非零值）。每个非零元存储一行一列一值。	易于构造和修改，用于矩阵组装。
CSR (Compressed Sparse Row)	三个数组：ptr（每行起始位置，长度 n+1）、col（列索引）、val（非零值）。按行压缩。	高效进行矩阵-向量乘法（行访问），是多数求解器的首选。
CSC (Compressed Sparse Column)	三个数组：ptr（每列起始位置）、row（行索引）、val（非零值）。按列压缩。	高效进行列操作，如分解时的列消去。

3. 稀疏直接求解器中的符号分解、数值分解和回代分别对应什么工作？

- 符号分解**：在不考虑数值的情况下，分析矩阵的稀疏结构，预测填充元位置，选择重排序策略，并分配存储空间。这一步只使用矩阵的模式（非零元位置），不进行浮点运算。
- 数值分解**：实际进行 LDL^T 或 LU 分解，计算数值因子（ L 、 D 的具体值）。利用符号分解确定的存储结

构，完成浮点计算。

- **回代**：利用已分解的因子求解不同右端项（前代 + 回代）。对于多载荷工况，此步骤可重复进行，效率极高。

4. 重排序为什么可以减少填充元并降低内存需求？

稀疏矩阵分解时，原本为零的位置可能产生新的非零元素（填充元）。填充元的数量取决于矩阵的非零元分布。通过重排序（如 AMD、RCM、METIS），可以重新排列自由度的顺序，使矩阵的非零元更加集中在主对角线附近（减小带宽），从而限制填充元的发生范围。实验表明，良好的重排序可将填充元数量降低一个数量级以上，进而显著减少内存和计算量。

5. 当矩阵规模继续增大时，直接求解器可能遇到哪些内存瓶颈？

直接求解器的主要瓶颈是**填充元爆炸**：虽然原矩阵是稀疏的，但分解后 L 和 U 的填充元可能导致内存需求急剧增长。对于三维问题，填充元数量通常为 $O(n^{4/3})$ 甚至 $O(n^2)$ ，当自由度达到数百万时，内存需求可能升至数百 GB 甚至 TB，超出单机内存。此时需要采用：

- 更高效的重排序算法（如 METIS）；
- 外存求解（out-of-core，将部分因子存储在硬盘）；
- 或改用迭代法（如预处理共轭梯度法）。

十二、说明

代码部分大部分由人工智能辅助完成，但已认真学习理解代码，并掌握本次作业的核心知识点和核心任务。

数组下标约定：本程序所有内部数组（节点编号、自由度编号、LM矩阵等）均采用 **0-based indexing**（从0开始）。输入JSON文件中的节点编号和自由度编号仍使用1-based，程序内部自动转换（见 `model.py` 和 `element.py` 中的 `-1` 操作）。